



UNIVERSITÀ DI PARMA

Le Espressioni

*We must either institute conventional forms
of expression or else pretend that we have
nothing to express.*

George Santayana, Soliloquies in England

- Operatori & Operandi
 - Tipologie
- Espressioni
- Valutazione
- Precedenza & Associatività
- Effetti collaterali

SUMMARY

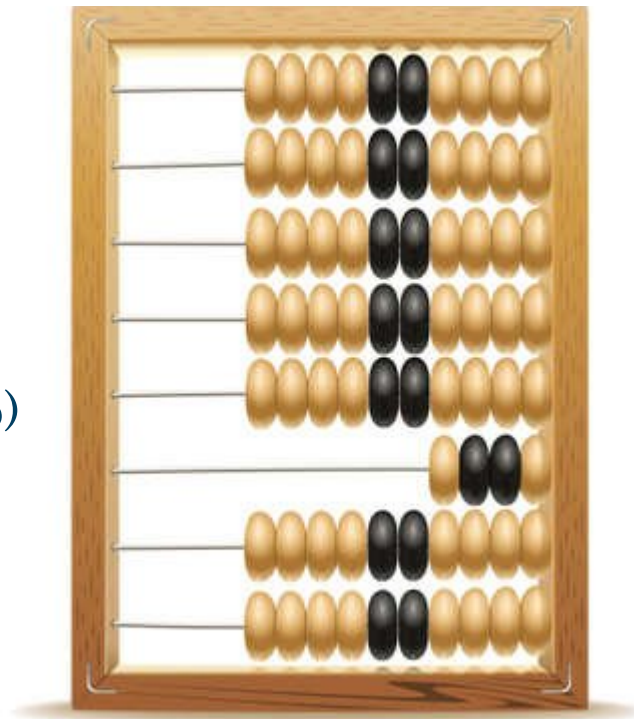


- **Expressions → compute a value**
 - They have a value (and a type)
 - They can be used wherever a value is expected
 - 17, a, b * c + 2, a > 0, a = 5 are all expressions
 - **Operators and Operands**
- **Statements → perform an action**
 - They are executed but do not return values
 - `return 0;` is a statement
 - `a=5;` is a statement as well!

$$A = B * C + D * 17 / K$$

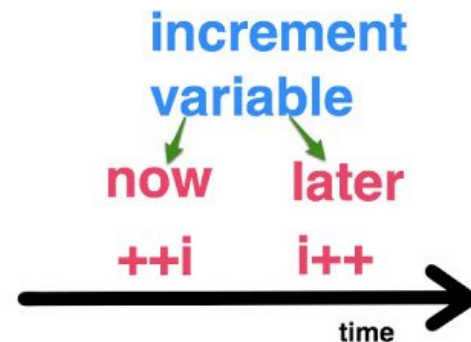
- Combinano uno o più operandi e permettono di valutare “operazioni”
 - Aritmetiche
 - Relazionali
 - Logiche
 - Bit a bit
 - Assegnamento
 - Condizionali e speciali
- A seconda del numero di operandi coinvolti:
 - Unari
 - Binari
 - Ternari

- Binari:
 - + → somma
 - - → sottrazione
 - * → moltiplicazione
 - / → divisione
 - % → resto della divisione intera (modulo)
- Unari:
 - + o - → segno
 - ++ → incremento di 1
 - -- → decremento di 1



- Permettono calcoli tra numeri interi o a virgola mobile
 - Eccetto %
- Attenzione a /
 - Se operandi interi
 - Divisione intera
 - $5/6 \rightarrow 0$
 - Se operando negativo problemi arrotondamento
 - $-9/7 \rightarrow -1$ oppure -2 *(C99 solved this)

- Operatori di incremento o decremento
 - Incremento variabile intera “a” di 1 $\rightarrow ++a$;
 - Decremento variabile intera “a” di 1 $\rightarrow --a$;
- Operazioni comuni
- **Caveat:** comportamento differente se prefisso o postfixo
 - $++a \neq a++$
 - Incrementano entrambi “a” ma se in espressioni:
 - Prefisso $\rightarrow$ eseguo incremento prima di valutare resto espressione
 - Postfisso $\rightarrow$ eseguo incremento dopo aver valutato resto espressione
 - If not sure, avoid them in expressions!



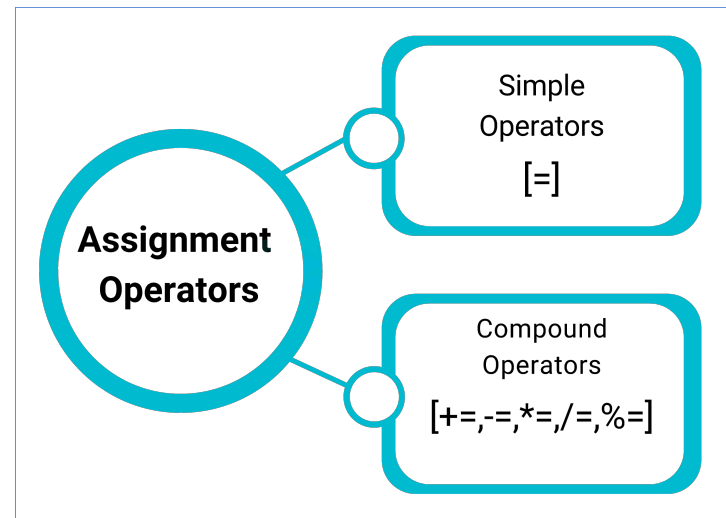
- Undefined behaviors

- `int x = i++ + i++;`
`// which i++ is evaluated first?`
- `printf("%d %d\n", i++, i++);`
`// evaluation order of args not defined`

- Safety rule: do not use multiple ++/-- for the same variable in the same expression

- `int x = i++;`
- `x = x + i++;` `// order of modifications is clear`

- Permettono assegnamento valore a variabile
- Semplice:
 - `=` → assegnamento
- Composti:
 - `+=` → assegnamento con somma
 - `-=` → ...
 - `*=`
 - `%=`
 - ...



- A differenza di altri operatori, gli assegnamenti hanno limiti su cosa posso avere come operandi
- A destra dell'operatore di assegnamento
 - Numeri, variabili, espressioni ecc.
 - Right Value o R-Value
- A sinistra dell'operatore di assegnamento
 - Solo variabili o comunque indirizzo memoria
 - Left Value o L-Value

- A volte necessità di valutare situazioni
 - Il numero x è pari o dispari?
 - a è più piccolo o più grande di b ?
- Vero o Falso, in C:
 - **Vero $\rightarrow$ qualunque numero diverso da 0**
 - **Falso $\rightarrow$ 0**

- $==$ $\rightarrow$ controllo uguaglianza (no virgola mobile!)
- $!=$ $\rightarrow$ controllo differenza (non esiste $\neq$)
- $>$ $\rightarrow$ piú grande di
- $<$ $\rightarrow$ piú piccolo di
- $>=$ $\rightarrow$ piú grande o eguale di
- $<=$ $\rightarrow$ piú piccolo o eguale di
- Hanno come risultato:
 - Vero $\rightarrow$ 1 (sempre 1!)
 - Falso $\rightarrow$ $\emptyset$



Name	Operator	Expression	Evaluates
Equals	==	5 == 5 K == 10	true depends
Not Equals	!=	50 != 25 100 != 100	true false
Less than	<	100 < 200 P < Q	true depends
Greater than	>	100 > 200 P > Q	false depends
Less than or equals	<=	25 <= 26 25 <= 25	true true
Greater than or equals	>=	1000 >= 1000 K >= (P + Q)	true depends

True → 1

- Operatori relazionali $\rightarrow$ test “semplici”
 - a è compreso nell’intervallo 10-18?
 - $a > 10$ ma anche $a < 18$!
- Gli operatori logici permettono di costruire test più complessi
- Combinando:
 - Espressioni che coinvolgono operatori relazionali
 - Espressioni che assumono valori vero/falso

- Operatori il cui risultato è vero(1)/falso(0)
- `&&` → and logico
- `||` → or logico
- `!` → not logico

<code>&&</code>	<code>!alive</code>	<code>alive</code>
<code>!dead</code>		
<code>dead</code>		

Operatori logici: tabelle di verità

a	b	a && b	a b	!a
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

- Combinazioni di operatori e operandi
- Esempi:
 - 17 → espressione senza operatori
 - $17 * 2 + 3$ → espressione costante
 - $a = b * 17 + c / 4 + e \% 5 * 2$ → calcolata in esecuzione

- Operatori differenti possono avere precedenze differenti
 - $3*4+2$
 - Nella valutazione delle espressioni si parte dagli operatori con maggiore precedenza
- A parità di precedenza gli operatori vengono valutati in base alla associatività
 - $a\%b\%c$ ($\rightarrow$)
 - $a = b = c;$ ($\leftarrow$)
- **Nel dubbio ()**

P
R
E
C
E
D
E
N
C
E



Operator	Description	Associativity
() [] . -> ++ --	Parentheses or function call, Brackets or array subscript, Dot or Member selection operator, Arrow operator, Postfix increment/decrement	left to right
++ -- + - ! ~ (type) * & sizeof	Prefix increment/decrement, Unary plus and minus, not operator and bitwise complement, type cast, Indirection or dereference operator, Address of operator, Determine size in bytes	right to left
* / %	Multiplication, division, and modulus	left to right
+ -	Addition and subtraction	left to right
<< >>	Bitwise left shift and right shift	left to right
< <= > >=	Relational less than/less than equal to, relational greater than/greater than or equal to	left to right
== !=	Relational equal to or not equal to	left to right
&	Bitwise AND	left to right
^	Bitwise exclusive OR	left to right
	Bitwise inclusive OR	left to right
&&	Logical AND	left to right
	Logical OR	left to right
?:	Ternary operator	right to left
= += -= *= /= %= &= ^= = <<= >>=	Assignment operator, Addition/subtraction assignment, Multiplication/division assignment, Modulus and bitwise assignment, Bitwise exclusive/inclusive OR assignment	right to left
,	Comma operator	left to right

- Precedence does not affect evaluation order
- As example with functions
 - $f() + g() * h()$
- The order used to invoke $f()$, $g()$, or $h()$ is not specified

- Condizionali
 - ?:
 - Unico operatore ternario del C
- Speciali
 - Li dettaglieremo man mano che li incontreremo
 - sizeof() dimensione di un dato
 - & indirizzo di una variabile
 - * puntatori o accesso memoria
 - , concatenazione di espressioni
 - [] indice array
 - ...

- Il C riesce ad operare anche a basso livello sui dati
 - Singolo bit
 - Dati interi
- Perché si usano?
 - Efficienza
 - Rappresentazione compatta di valori vero/falso ovvero 1/0
 -

- Lavorano a livello dei singoli bit di ciascun dato
 - $\&$ $\rightarrow$ and bit a bit
 - $|$ $\rightarrow$ or bit a bit
 - $\sim$ $\rightarrow$ not bit a bit
 - $\wedge$ $\rightarrow$ xor bit a bit
 - $\ll$ $\rightarrow$ left shift
 - $\gg$ $\rightarrow$ right shift
- Li vedremo nel dettaglio successivamente

- `=` instead of `==`
 - Some compilers issue a warning
 - Not so easy to spot!
- Interval match
 - `10 < a < 15` looks correct from a mathematical point of view
 - Actually evaluated as `(10 < a) < 15` namely as `1 < 15` or `0 < 15` → always true
- Approximations
 - `0.1 + 0.2 == 0.3` → actually false
- Integer division
 - `5/2` → 2 and not 2.5



UNIVERSITÀ DI PARMA

Le Espressioni



KEEP
CALM
IT'S
QUESTION
TIME

We must either institute conventional forms of expression or else pretend that we have nothing to express.

George Santayana, Soliloquies in England